

Chain of Thought Prompting

Teach the AI to *think out loud* — step by step

A plain-English guide with Python examples

What is Chain of Thought Prompting?

Normally, when you ask an AI a question, it jumps straight to the answer. That works fine for simple questions. But for complex problems — like maths, logic puzzles, or multi-step analysis — jumping straight to an answer often leads to mistakes.

Chain of Thought (CoT) prompting is a technique where you ask the AI to *show its reasoning* before giving the final answer. Instead of just saying the answer, the AI walks through each step — like a student showing their working in an exam.

Think of it this way: you wouldn't trust a calculator that only showed '42' with no formula. CoT prompting makes the AI show the formula, not just the answer.

Without CoT vs. With CoT

Without CoT (Risky)	With CoT (Safe)
User: "If a train travels 60 km/h for 2.5 hours, how far does it go?" AI: "150 km" # No reasoning shown. If the answer # were wrong, you'd have no idea why.	User: "If a train travels 60 km/h for 2.5 hours, how far does it go? Think step by step." AI: "Speed = 60 km/h Time = 2.5 hours Distance = Speed x Time = 60 x 2.5 = 150 km"

1. How Does It Work?

CoT prompting works by adding a simple instruction to your prompt that tells the AI to reason before concluding. You don't need any special tools or APIs — just the right words.

Common trigger phrases:

- "Think step by step."
- "Let's reason through this carefully."
- "Show your working before giving the final answer."
- "Break this problem down into steps."
- "Explain your reasoning, then conclude."

Tip: 'Think step by step' is the most widely tested phrase in research. It is a good default for most tasks.

2. Types of Chain of Thought Prompting

A. Zero-Shot CoT

You do not provide any examples. You simply add a reasoning instruction to your prompt. The AI figures out how to reason on its own.

```
import anthropic

client = anthropic.Anthropic()

# Zero-Shot CoT: no examples, just the magic phrase
response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=512,
    messages=[{
        "role": "user",
        "content": (
            "A bookstore sells books for $12 each. "
            "If you buy 3 or more, you get a 10% discount on all books. "
            "How much do 5 books cost? "
            "Think step by step." # <-- Zero-Shot CoT trigger
        )
    }],
)
print(response.content[0].text)

# AI will reason:
# Step 1: Normal price = 5 x $12 = $60
# Step 2: 5 >= 3, so discount applies
# Step 3: 10% of $60 = $6
# Step 4: Final price = $60 - $6 = $54
```

B. Few-Shot CoT

You provide one or more worked examples before asking the real question. This teaches the AI the exact style of reasoning you want.

```
import anthropic

client = anthropic.Anthropic()

# Few-Shot CoT: show the AI HOW to reason with examples
few_shot_prompt = '''
Example 1:
Q: A store has 20 apples. They sell 8. Then they get 5 more. How many apples?
A: Start with 20 apples.
   Sell 8:  $20 - 8 = 12$  apples.
   Get 5 more:  $12 + 5 = 17$  apples.
   Answer: 17 apples.

Example 2:
Q: A jar has 30 sweets. 3 children each take 4 sweets. How many are left?
A: Each child takes 4 sweets. 3 children:  $3 \times 4 = 12$  sweets taken.
   Remaining:  $30 - 12 = 18$  sweets.
   Answer: 18 sweets.

Now answer this:
Q: A parking lot has 50 spaces. 18 cars are parked.
   Then 7 cars leave and 12 new cars arrive. How many free spaces are left?
'''

response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=512,
    messages=[{"role": "user", "content": few_shot_prompt}],
)
print(response.content[0].text)

# AI will follow the same step-by-step style as the examples.
```

C. Self-Consistency CoT

Run the same CoT prompt multiple times and pick the most common answer. This reduces errors by using a 'majority vote' across several reasoning paths.

```
import anthropic
from collections import Counter

client = anthropic.Anthropic()

problem = (
    "Emma has twice as many stickers as Liam. Together they have 48 stickers. "
    "How many does Emma have? Think step by step."
)

answers = []

# Run the same prompt 3 times (use more runs for higher accuracy)
for i in range(3):
    response = client.messages.create(
        model="claude-opus-4-6",
        max_tokens=300,
        messages=[{"role": "user", "content": problem}],
    )
    reply = response.content[0].text
    answers.append(reply)
    print(f"Run {i+1}: {reply[:80]}...")

# In practice: extract the final number from each reply and majority-vote
# Here we just print all answers for inspection
print("\nAll answers:", answers)

# Tip: with temperature > 0, each run may reason differently
# but usually converges on the same correct final answer.
```

Self-Consistency CoT is most useful when accuracy is critical, such as in medical triage, financial calculations, or legal reasoning. It is slower and costs more tokens, so use it selectively.

3. When Should You Use Chain of Thought?

CoT prompting shines on tasks that require multiple steps, conditional logic, or careful analysis. It is **not needed** for simple factual questions ('What is the capital of France?') — there, it just wastes tokens.

Use Case	Why CoT Helps	Trigger Phrase
Maths & Arithmetic	Multi-step calculations are error-prone without reasoning	"Think step by step."
Logic & Puzzles	Deductive reasoning needs ordered inference	"Reason through this carefully."
Code Debugging	Finding bugs requires tracing execution flow	"Walk through the code line by line."
Medical / Legal Analysis	High-stakes decisions need auditable reasoning	"Explain your reasoning before concluding."
Financial Modelling	Compound calculations need each step verified	"Show all calculations step by step."
Essay / Argument Planning	Structured writing needs logical flow	"Break your argument into steps."
Data Interpretation	Drawing conclusions from data needs reasoning	"Analyse each part, then summarise."
Decision Making	Weighing pros and cons needs explicit comparison	"Think through each option before deciding."

4. Real-World Code Examples

Example A — Maths Word Problem

```
import anthropic

client = anthropic.Anthropic()

def solve_math(problem: str) -> str:
    """Solve a maths problem using Zero-Shot CoT."""
    response = client.messages.create(
        model="claude-opus-4-6",
        max_tokens=600,
        system=(
            "You are a maths tutor. Always show your working step by step. "
            "End your answer with: Final Answer: <number>"
        ),
        messages=[{
            "role": "user",
            "content": f"{problem}\n\nThink step by step."
        }],
    )
    return response.content[0].text

# Test it
problem = (
    "A factory produces 240 widgets per hour. It runs 8 hours on weekdays and 5 hours "
    "on weekends. How many widgets are produced in a week (5 weekdays + 2 weekend days)? "
)
print(solve_math(problem))

# Expected reasoning:
# Weekday production   = 240 x 8 = 1920 widgets/day
# Weekend production   = 240 x 5 = 1200 widgets/day
# Total weekdays       = 1920 x 5 = 9600
# Total weekend days    = 1200 x 2 = 2400
# Grand total          = 9600 + 2400 = 12000 widgets
# Final Answer: 12000
```

Example B — Code Debugging with CoT

```

import anthropic

client = anthropic.Anthropic()

buggy_code = '''
def find_average(numbers):
    total = 0
    for n in numbers:
        total = total + n
    return total / len(numbers)

print(find_average([]))  # This crashes!
'''

response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=600,
    system=(
        "You are a Python debugging expert. "
        "Walk through the code line by line, identify the bug, "
        "explain why it fails, then provide the fixed code."
    ),
    messages=[{
        "role": "user",
        "content": f"Debug this code:\n\n{buggy_code}"
    }],
)
print(response.content[0].text)

# AI reasoning:
# Line 1-4: Loops correctly and sums numbers. OK.
# Line 5: Divides total by len(numbers).
# Problem: If numbers is empty, len([]) = 0 -> ZeroDivisionError!
# Fix: Check for empty list before dividing.

```

Example C — Business Decision Making

```
import anthropic

client = anthropic.Anthropic()

scenario = '''
A startup has $50,000 left in the bank. They have two options:
Option A: Spend $40,000 on a marketing campaign expected to bring
          $80,000 in new revenue over 3 months.
Option B: Hire a developer for $35,000 to build a new feature
          expected to increase retention by 20%, worth ~$60,000/year.
Which option should they choose?
'''

response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=700,
    system=(
        "You are a startup financial advisor. "
        "Break down the decision step by step: analyse cash flow, "
        "ROI, risk, and timeline for each option before recommending."
    ),
    messages=[{
        "role": "user",
        "content": f"{scenario}\n\nReason through each option carefully."
    }],
)
print(response.content[0].text)
```


Example D — Structured CoT Using XML Tags (Advanced)

You can ask the AI to put its reasoning inside tags. This makes it easy to programmatically extract just the final answer, separate from the thinking.

```
import anthropic
import re

client = anthropic.Anthropic()

def structured_cot(question: str) -> dict:
    """Returns both the reasoning and the final answer separately."""
    response = client.messages.create(
        model="claude-opus-4-6",
        max_tokens=700,
        system=(
            "Answer questions by first reasoning inside <thinking> tags, "
            "then giving only the final answer inside <answer> tags."
        ),
        messages=[{
            "role": "user",
            "content": question
        }],
    )
    text = response.content[0].text

    # Extract reasoning and answer separately
    thinking = re.search(r"<thinking>(.*?)</thinking>", text, re.DOTALL)
    answer = re.search(r"<answer>(.*?)</answer>", text, re.DOTALL)

    return {
        "reasoning": thinking.group(1).strip() if thinking else "",
        "answer": answer.group(1).strip() if answer else text,
    }

result = structured_cot(
    "If you invest $1000 at 5% annual interest, compounded yearly, "
    "how much do you have after 3 years?"
)
print("Reasoning:", result["reasoning"])
print("Answer:", result["answer"])
```

Using XML tags like <thinking> and <answer> is a best practice when building AI applications. Your code can extract just the final answer for display, while logging the full reasoning for debugging — without the user seeing a wall of text.

5. What a CoT Conversation Looks Like

Here is an annotated conversation showing CoT reasoning in action for a logic puzzle.

USER	There are 3 boxes. One has only apples, one has only oranges, and one has both. All boxes are labelled WRONG. You can pick one fruit from one box. How do you figure out which box is which? Think step by step.
ASSISTANT (Step 1)	Step 1 — What do we know? All labels are WRONG, so the box labelled 'Apples' does NOT contain only apples. Same for the other labels.
ASSISTANT (Step 2)	Step 2 — Key insight: Pick from the box labelled 'Apples + Oranges'. Since its label is wrong, it cannot contain both. So it must be either apples only or oranges only.
ASSISTANT (Step 3)	Step 3 — Say you pull out an apple. Then that box is 'Apples only'. Now the box labelled 'Oranges' cannot be oranges — and it can't be apples (taken). So it must be 'Apples + Oranges'.
ASSISTANT (Answer)	Final Answer: The remaining box labelled 'Apples' must be 'Oranges only'. One pick from the mixed-label box lets you correctly identify all three.

6. Do's and Don'ts

✓ DO use CoT for multi-step problems	✗ DON'T use CoT for simple factual lookups
✓ DO combine CoT with a system prompt for consistent format	✗ DON'T ask for step-by-step on one-word answer tasks
✓ DO use XML tags to separate reasoning from the final answer	✗ DON'T blindly trust the reasoning — verify critical outputs
✓ DO use Few-Shot CoT when you need a specific reasoning style	✗ DON'T use Self-Consistency CoT on every call — it's expensive
✓ DO tell the AI to end with 'Final Answer:' for easy parsing	✗ DON'T make the CoT instruction so long it buries the actual question

Chain of Thought prompting is one of the simplest yet most powerful techniques in prompt engineering. By asking the AI to reason out loud, you get more accurate answers, auditable logic,

and the ability to catch mistakes before they matter.

Start with 'Think step by step.' — and work from there.